GROUPE ISI	**Institut Supérieur Informatique**	
	TP 2 : Modules et imports	Prof. M. Djigo Période : 2026 – 2027
	Classe(s) : Licence 2 *IAGE* – Séance 2	Durée : 1 h

Consignes générales.

- Créez un projet PyCharm nommé `tp2`, puis **un dossier par exercice** : `exo1/`, `exo2/`, ...
- Dans chaque dossier, **un seul fichier est exécuté** : `main.py` (clic droit sur `main.py` puis *Run*). Les autres fichiers sont des **modules** : ils contiennent uniquement des constantes et des fonctions, et on ne les lance jamais.
- Créez les fichiers **dans l'ordre indiqué**, et respectez exactement les **noms de fichiers et de fonctions** donnés.
- Une fonction qui **renvoie** une valeur utilise `return` ; une fonction qui **affiche** utilise `print`.
- Comparez votre affichage avec l'**exemple d'exécution** de chaque exercice. Les nombres tirés au hasard et les dates seront différents chez vous : c'est normal.
- Utilisez uniquement des **listes** et des **dictionnaires**. On suppose que l'utilisateur saisit des valeurs du bon type. La fiche utilitaire des modules résume `datetime`, `random` et `math`.

Exercice 1 : Le carnet de crédit de la boutique

Contexte. Modou tient une boutique de quartier. Beaucoup de clients prennent des produits à crédit et paient en fin de mois ; jusqu'ici, Modou notait tout dans un cahier. Il veut un petit programme : la liste des clients, la liste des produits et la gestion du carnet sont rangées dans trois modules différents.

À savoir

- Acheter **à crédit**, c'est prendre la marchandise maintenant et payer plus tard. La **dette** d'un client est la somme qu'il doit encore à la boutique.
- Un **remboursement** diminue la dette. On ne peut pas rembourser plus que ce que l'on doit.
- Le **plafond** est la dette maximale autorisée par client : ici 15 000 FCFA. Une vente à crédit qui ferait dépasser ce plafond est refusée.

```
exo1/
    clients.py      (créé en 1er)
    produits.py     (2e)
    carnet.py       (3e)
    main.py         (4e, le seul exécuté)
```

1. **Fichier `clients.py`**

   Définir le dictionnaire des clients, dont la valeur est la dette de chacun en FCFA :

   ```
   CLIENTS = {"Awa": 0, "Moussa": 2500, "Fatou": 7000, "Ibou": 0}
   ```

2. **Fichier `produits.py`**

   Définir le dictionnaire des produits et leur prix à l'unité (le riz et le sucre sont vendus au kilo) :

```
PRODUITS = {"riz": 600, "sucre": 700, "huile": 1400,
            "lait": 250, "pain": 150, "savon": 300}
```

Écrire la fonction `afficher_produits()` qui affiche `--- Produits ---` puis une ligne par produit : `riz :  600 FCFA`.

3. **Fichier** `carnet.py`

   Importer `CLIENTS` depuis `clients` et `PRODUITS` depuis `produits` (avec `from ... import`). Définir `PLAFOND = 15000`, puis écrire :

   - `prendre_a_credit(client, produit, quantite)` : afficher `Client inconnu` si le client n'est pas une clé de `CLIENTS`, `Produit inconnu` si le produit n'est pas une clé de `PRODUITS`. Sinon, calculer le montant (prix × quantité) : si la nouvelle dette dépasserait `PLAFOND`, afficher le refus ; sinon, ajouter le montant à la dette et afficher la nouvelle dette ;
   - `rembourser(client, montant)` : si le montant est supérieur à la dette, afficher une erreur ; sinon, diminuer la dette et afficher ce qu'il reste à payer ;
   - `afficher_carnet()` : afficher la dette de chaque client, puis le **total dû** à la boutique (somme de toutes les dettes).

4. **Fichier** `main.py`

   Importer le module `carnet` avec l'**alias** `cr`, et `afficher_produits` avec `from produits import ...`. Puis :

   a) afficher `=== Boutique de Modou ===` et le menu `1. Vente à crédit 2. Remboursement 3. Carnet 0. Quitter :` ;

   b) tant que le choix est différent de `"0"` : `"1"` affiche les produits, demande le client, le produit et la quantité, puis appelle `cr.prendre_a_credit` ; `"2"` demande le client et le montant, puis appelle `cr.rembourser` ; `"3"` appelle `cr.afficher_carnet()` ; tout autre choix affiche `Choix invalide` ; puis on réaffiche le menu ;

   c) à la sortie de la boucle, afficher `Fermeture de la boutique.`

Exemple d'exécution attendu (ce que tape l'utilisateur est écrit après les deux-points) :

```
=== Boutique de Modou ===
1. Vente à crédit  2. Remboursement  3. Carnet  0. Quitter : 1
--- Produits ---
riz : 600 FCFA
(... autres produits ...)
Client : Awa
Produit : riz
Quantité : 2
2 riz = 1200 FCFA. Awa doit maintenant 1200 FCFA
1. Vente à crédit  2. Remboursement  3. Carnet  0. Quitter : 1
(... liste des produits ...)
Client : Fatou
Produit : huile
Quantité : 6
Crédit refusé : le plafond de 15000 FCFA serait dépassé
1. Vente à crédit  2. Remboursement  3. Carnet  0. Quitter : 2
Client : Moussa
Montant remboursé : 3000
Erreur : Moussa ne doit que 2500 FCFA
1. Vente à crédit  2. Remboursement  3. Carnet  0. Quitter : 2
Client : Moussa
Montant remboursé : 1000
Merci ! Moussa doit encore 1500 FCFA
```

```
1. Vente à crédit  2. Remboursement  3. Carnet  0. Quitter : 3
--- Carnet de crédit ---
Awa : 1200 FCFA
Moussa : 1500 FCFA
Fatou : 7000 FCFA
Ibou : 0 FCFA
Total dû à la boutique : 9700 FCFA
1. Vente à crédit  2. Remboursement  3. Carnet  0. Quitter : 0
Fermeture de la boutique.
```

Exercice 2 : Suivi d'un poulailler de poulets de chair

Contexte. Un jeune éleveur de Keur Massar a reçu 500 poussins. Chaque semaine, il compte les poulets morts et veut savoir où en est sa bande : combien de poulets il lui reste, s'il doit s'inquiéter de la mortalité, combien de sacs d'aliment acheter, et quel bénéfice il peut espérer à la vente. Les informations sur la bande, l'alimentation et la vente sont dans trois modules.

À savoir

- Une **bande** est un lot de poussins arrivés le même jour. L'**effectif** est le nombre de poulets encore vivants : effectif initial − morts.
- Le **taux de mortalité** est le pourcentage de poulets morts : morts / effectif initial × 100. Au-delà de 5 %, l'éleveur doit appeler le vétérinaire.
- Les poulets de chair sont vendus vers **45 jours**. L'âge de la bande se calcule avec les dates : `(date.today() - date_arrivee).days`.
- Valeurs simplifiées : un poulet consomme environ 4 kg d'aliment pendant tout l'élevage ; l'aliment est vendu en sacs de 50 kg à 17 500 FCFA ; un poussin coûte 650 FCFA et un poulet se vend 3 500 FCFA.
- **Bénéfice** = recette de la vente − dépenses (achat des poussins + aliment).

```
exo2/
    bande.py           (1er)
    alimentation.py    (2e)
    ventes.py          (3e)
    main.py            (4e, le seul exécuté)
```

1. **Fichier** `bande.py`

   Importer `date` depuis `datetime`, puis définir la bande :

   ```
   BANDE = {"nom": "Bande d'octobre", "date_arrivee": date(2026, 9, 20),
            "effectif_initial": 500, "morts": 0, "prix_poussin": 650}
   ```

   Écrire les fonctions :

   - `enregistrer_morts(nb)` : ajoute `nb` au nombre de morts ;
   - `effectif()` : **renvoie** le nombre de poulets vivants ;
   - `taux_mortalite()` : **renvoie** le taux de mortalité arrondi à 1 décimale (12 morts sur 500 donnent `2.4`) ;
   - `age_jours()` : **renvoie** l'âge de la bande en jours.

2. **Fichier** `alimentation.py`

   Importer `math`. Définir `CONSO_PAR_POULET = 4`, `POIDS_SAC = 50` et `PRIX_SAC = 17500`, puis :

- `sacs_necessaires(nb_poulets)` : **renvoie** le nombre de sacs à acheter, arrondi au sac supérieur avec `math.ceil`. Pour 488 poulets : $488 \times 4/50 = 39{,}04$, soit **40 sacs** ;
- `cout_aliment(nb_poulets)` : **renvoie** le prix de ces sacs.

3. **Fichier** `ventes.py`

   Définir `PRIX_VENTE = 3500` et `AGE_VENTE = 45`, puis :

   - `jours_avant_vente(age)` : **renvoie** le nombre de jours qui restent avant 45 jours, ou 0 si la bande a déjà 45 jours ou plus ;
   - `recette(nb_poulets)` : **renvoie** l'argent rapporté par la vente de tous les poulets.

4. **Fichier** `main.py`

   Importer le module `bande` avec `import bande`, les deux fonctions de `alimentation` avec `from ... import`, et le module `ventes` avec l'alias `vt`. Puis :

   a) afficher le nom de la bande et sa date d'arrivée au format `jour/mois/année` (lire le dictionnaire avec `bande.BANDE[...]`) ;

   b) demander le nombre de poulets morts cette semaine et l'enregistrer ;

   c) afficher l'effectif et le taux de mortalité ; si le taux dépasse 5, afficher l'alerte ;

   d) afficher l'âge de la bande et le nombre de jours avant la vente ;

   e) afficher le nombre de sacs d'aliment et leur coût (pour l'effectif actuel) ;

   f) calculer et afficher les dépenses (effectif initial × prix d'un poussin + coût de l'aliment), la recette prévue et le bénéfice prévu.

Exemple d'exécution attendu (ce que tape l'utilisateur est écrit après les deux-points) :

```
=== Bande d'octobre ===
Arrivée des poussins : 20/9/2026
Nombre de poulets morts cette semaine : 12
Effectif actuel : 488 poulets
Taux de mortalité : 2.4 %
Âge de la bande : 37 jours, vente dans 8 jours
Aliment nécessaire : 40 sacs de 50 kg (700000 FCFA)
Dépenses : 1025000 FCFA
Recette prévue : 1708000 FCFA
Bénéfice prévu : 683000 FCFA
```

Avec 30 morts, le taux vaut 6.0 % et le programme affiche en plus : `ALERTE : mortalité trop élevée, appelez le vétérinaire !`

Exercice 3 : Production d'œufs d'un poulailler de pondeuses

Contexte. Une ferme de Sangalkam élève 300 poules pondeuses. Chaque jour, l'ouvrier ramasse les œufs, met de côté ceux qui sont cassés et range les autres en plateaux de 30 pour les vendre aux boutiques. Le gérant veut simuler une semaine de production et obtenir un rapport. Le ramassage est simulé au hasard.

À savoir

- Le **taux de ponte** indique la part des poules qui ont pondu : œufs ramassés / nombre de poules × 100. 270 œufs pour 300 poules donnent 90 %.
- Les œufs cassés ne sont pas vendus. Les œufs **vendables** sont rangés en **plateaux de 30** ; un plateau se vend 2 500 FCFA. Les œufs qui ne remplissent pas un plateau complet restent à la ferme.
- `oeufs // 30` donne le nombre de plateaux complets ; `oeufs % 30` donne le nombre d'œufs restants. Exemple : 1797 œufs font 59 plateaux et il reste 27 œufs.

```
exo3/
    poules.py      (1er)
    ponte.py       (2e)
    plateaux.py    (3e)
    rapport.py     (4e)
    main.py        (5e, le seul exécuté)
```

1. **Fichier** `poules.py`

   Définir la constante `NB_POULES = 300`.

2. **Fichier** `ponte.py`

   Importer `random`, et `NB_POULES` depuis `poules`. Écrire :

   - `ramassage_du_jour()` : **renvoie** un nombre d'œufs au hasard entre 220 et 280 ;
   - `oeufs_casses()` : **renvoie** un nombre d'œufs cassés au hasard entre 0 et 10 ;
   - `taux_ponte(oeufs)` : **renvoie** le taux de ponte arrondi à 1 décimale.

3. **Fichier** `plateaux.py`

   Définir `TAILLE_PLATEAU = 30` et `PRIX_PLATEAU = 2500`, puis écrire `nb_plateaux(oeufs)` et `oeufs_restants(oeufs)` qui **renvoient** le nombre de plateaux complets et le nombre d'œufs restants.

4. **Fichier** `rapport.py`

   Importer `taux_ponte` depuis `ponte`, et `nb_plateaux`, `oeufs_restants`, `PRIX_PLATEAU` depuis `plateaux`. La semaine est une **liste de dictionnaires** : `[{"jour": "Lundi", "ramasses": 272, "casses": 1}, ...]`. Écrire :

   - `meilleur_jour(semaine)` : parcourt la liste et **renvoie** le nom du jour où l'on a ramassé le plus d'œufs ;
   - `afficher_rapport(semaine)` : calcule le total des œufs ramassés et cassés, puis affiche : œufs ramassés, cassés, vendables, nombre de plateaux et œufs restants, recette, taux de ponte moyen (taux de ponte de la moyenne quotidienne : total ramassé / 7) et meilleur jour.

5. **Fichier** `main.py`

   Importer `ramassage_du_jour`, `oeufs_casses` et `taux_ponte` avec `from ponte import ...`, et le module `rapport` avec `import rapport`. Puis :

a) définir la liste `JOURS = ["Lundi", "Mardi", ..., "Dimanche"]` et une liste vide `semaine` ;

b) pour chaque jour : tirer le ramassage et les œufs cassés, ajouter le dictionnaire du jour à `semaine` avec `.append`, et afficher la ligne du jour avec son taux de ponte ;

c) appeler `rapport.afficher_rapport(semaine)`.

Exemple d'exécution attendu (ce que tape l'utilisateur est écrit après les deux-points) :

```
=== Poulailler de pondeuses : 300 poules ===
Lundi : 272 œufs ramassés, 1 cassés, taux de ponte 90.7 %
Mardi : 241 œufs ramassés, 1 cassés, taux de ponte 80.3 %
Mercredi : 265 œufs ramassés, 7 cassés, taux de ponte 88.3 %
Jeudi : 263 œufs ramassés, 10 cassés, taux de ponte 87.7 %
Vendredi : 272 œufs ramassés, 6 cassés, taux de ponte 90.7 %
Samedi : 242 œufs ramassés, 5 cassés, taux de ponte 80.7 %
Dimanche : 280 œufs ramassés, 8 cassés, taux de ponte 93.3 %
===== Rapport de la semaine =====
Œufs ramassés : 1835
Œufs cassés : 38
Œufs vendables : 1797
Plateaux de 30 : 59 (reste 27 œufs)
Recette : 147500 FCFA
Taux de ponte moyen : 87.4 %
Meilleur jour : Dimanche
```

Exercice 4 : Le marché aux moutons de la Tabaski

Contexte. Un éleveur installe son enclos de moutons quelques mois avant la Tabaski. Les clients choisissent un mouton, versent une avance pour le réserver, et paient le reste avant la fête. L'éleveur veut afficher les moutons encore disponibles avec leur prix, enregistrer les réservations et connaître à tout moment l'argent encaissé et celui qui reste à recevoir.

À savoir

- Le prix d'un mouton dépend de sa **race** et de son **poids** : prix = poids (kg) × prix du kilo de sa race. Prix simplifiés : Bali-Bali 3 500 FCFA/kg, Touabire 4 000 FCFA/kg, Ladoum 6 000 FCFA/kg.
- L'**avance** est la somme versée pour réserver : elle doit être au moins égale à **30 %** du prix. Le **reste à payer** est prix − avance.
- On suppose que la Tabaski 2027 tombe le 16 mai 2027. Le nombre de jours restants est `(TABASKI - date.today()).days`.
- `if not mouton["vendu"]:` signifie "si le mouton n'est pas vendu".

```
exo4/
    troupeau.py     (1er)
    tarifs.py       (2e)
    vente.py        (3e)
    main.py         (4e, le seul exécuté)
```

1. **Fichier `troupeau.py`**

   Recopier le dictionnaire des moutons (code du mouton : fiche du mouton) :

```python
MOUTONS = {
    "M1": {"race": "Touabire", "poids": 45, "vendu": False},
    "M2": {"race": "Bali-Bali", "poids": 40, "vendu": False},
    "M3": {"race": "Ladoum", "poids": 60, "vendu": False},
    "M4": {"race": "Touabire", "poids": 52, "vendu": False},
```

```
    "M5": {"race": "Bali-Bali", "poids": 48, "vendu": False},
    "M6": {"race": "Ladoum", "poids": 75, "vendu": False},
}
```

2. **Fichier** `tarifs.py`

   Importer `MOUTONS` depuis `troupeau`. Définir `PRIX_KG = {"Bali-Bali": 3500, "Touabire": 4000, "Ladoum": 6000}`, puis :

   - `prix_mouton(code)` : **renvoie** le prix du mouton. `prix_mouton("M3")` renvoie `360000` ($60 \times 6\,000$) ;
   - `afficher_disponibles()` : affiche uniquement les moutons non vendus : `M1 : Touabire, 45 kg, 180000 FCFA`.

3. **Fichier** `vente.py`

   Importer `date`, `MOUTONS` et `prix_mouton`. Définir `TABASKI = date(2027, 5, 16)`, `TAUX_AVANCE = 0.30` et une liste vide `VENTES`, puis :

   - `jours_avant_tabaski()` : **renvoie** le nombre de jours avant la fête ;
   - `vendre(code, client, avance)` : afficher `Code inconnu` ou `Ce mouton est déjà vendu` si besoin. Sinon, calculer l'avance minimale (`int(prix * TAUX_AVANCE)`) : si l'avance est trop faible, afficher le minimum ; sinon, marquer le mouton comme vendu, ajouter à `VENTES` le dictionnaire `{"code": ..., "client": ..., "prix": ..., "avance": ...}` et afficher le reçu ;
   - `afficher_ventes()` : affiche chaque vente, puis le nombre de moutons vendus, l'argent encaissé (somme des avances) et le reste à encaisser (somme des restes à payer).

4. **Fichier** `main.py`

   Importer `afficher_disponibles` avec `from tarifs import ...`, et le module `vente` avec l'alias `vt`. Puis :

   a) afficher le titre, le nombre de jours avant la Tabaski et les moutons disponibles ;

   b) demander un code de mouton ; tant qu'il est différent de `fin`, demander le nom du client et l'avance, appeler `vt.vendre`, réafficher les moutons disponibles et redemander un code ;

   c) afficher le bilan avec `vt.afficher_ventes()`.

Exemple d'exécution attendu (ce que tape l'utilisateur est écrit après les deux-points) :

```
=== Marché aux moutons de la Tabaski ===
Tabaski dans 201 jours
--- Moutons disponibles ---
M1 : Touabire, 45 kg, 180000 FCFA
M2 : Bali-Bali, 40 kg, 140000 FCFA
M3 : Ladoum, 60 kg, 360000 FCFA
M4 : Touabire, 52 kg, 208000 FCFA
M5 : Bali-Bali, 48 kg, 168000 FCFA
M6 : Ladoum, 75 kg, 450000 FCFA
Code du mouton (fin pour terminer) : M3
Nom du client : Ousmane
Avance versée : 50000
Avance insuffisante : minimum 108000 FCFA
(... liste des moutons ...)
Code du mouton (fin pour terminer) : M3
Nom du client : Ousmane
Avance versée : 120000
Reçu : Ousmane réserve M3 pour 360000 FCFA. Avance 120000 FCFA, reste 240000 FCFA
```

```
(... liste des moutons, sans M3 ...)
Code du mouton (fin pour terminer) : M1
Nom du client : Awa
Avance versée : 100000
Reçu : Awa réserve M1 pour 180000 FCFA. Avance 100000 FCFA, reste 80000 FCFA
(... liste des moutons, sans M3 ni M1 ...)
Code du mouton (fin pour terminer) : fin
===== Bilan des ventes =====
M3 - Ousmane : 360000 FCFA (avance 120000 FCFA)
M1 - Awa : 180000 FCFA (avance 100000 FCFA)
Moutons vendus : 2
Argent encaissé : 220000 FCFA
Reste à encaisser avant la Tabaski : 320000 FCFA
```

Exercice 5 : La caisse d'une supérette (paquet)

Contexte. La supérette Baraka remplace sa calculatrice par un logiciel de caisse. Le logiciel gère le stock, la caisse, une carte de fidélité et un rapport de fin de journée. Il est organisé comme une application Django : un **paquet** `superette` dont chaque module a un rôle précis, et un seul fichier `main.py` qui les fait travailler ensemble.

À savoir

- Le **stock** est la quantité disponible de chaque produit. Chaque vente le diminue ; on ne peut pas vendre plus que le stock. Un produit dont il reste moins de 5 unités est **à commander** auprès du fournisseur.
- Le **ticket de caisse** indique pour chaque ligne la quantité, le produit et le sous-total (prix × quantité), puis le total.
- **Carte de fidélité** : le client gagne 1 point par tranche de 1 000 FCFA payés (`montant // 1000`). Avec 100 points, il peut obtenir 1 000 FCFA de réduction ; les 100 points sont alors retirés de sa carte.
- **Chiffre d'affaires** : somme de toutes les ventes de la journée. **Panier moyen** : chiffre d'affaires / nombre de clients.
- Entre modules d'un même paquet, on importe avec un point : `from .stock import STOCK`. Le dossier `superette` doit contenir un fichier `__init__.py` vide.

```
exo5/
    main.py              (le seul exécuté, créé en dernier)
    superette/
        __init__.py      (fichier vide)
        stock.py
        caisse.py
        fidelite.py
        rapport.py
```

1. **Fichier** `superette/stock.py`

   Recopier le stock (produit : fiche du produit) et définir `SEUIL_ALERTE = 5` :

   ```
   STOCK = {
       "riz": {"prix": 4500, "quantite": 20},
       "huile": {"prix": 1600, "quantite": 12},
       "sucre": {"prix": 700, "quantite": 30},
       "lait": {"prix": 2750, "quantite": 6},
       "savon": {"prix": 300, "quantite": 40},
       "pates": {"prix": 450, "quantite": 4},
   }
   ```

Écrire `afficher_stock()` (une ligne par produit avec son prix et sa quantité), `disponible(produit, quantite)` qui **renvoie** `True` si le stock suffit, `retirer(produit, quantite)` qui diminue le stock, et `a_commander()` qui **renvoie** la liste des produits dont la quantité est inférieure à `SEUIL_ALERTE`.

2. **Fichier** `superette/caisse.py`

   Importer `STOCK`, `disponible` et `retirer` avec l'import relatif `from .stock import ...`. Le panier est une liste de dictionnaires {`"produit": ..., "prix": ..., "quantite": ...`}. Écrire :

   - `ajouter_article(panier, produit, quantite)` : afficher `Produit inconnu` si le produit n'existe pas, `Stock insuffisant : il reste 4 pates` si le stock ne suffit pas ; sinon, retirer la quantité du stock et ajouter la ligne au panier ;
   - `total(panier)` : **renvoie** le total du panier ;
   - `afficher_ticket(panier)` : affiche le ticket comme dans l'exemple.

3. **Fichier** `superette/fidelite.py`

   Recopier les cartes (numéro de téléphone : fiche du client) :

   ```
   CARTES = {
       "771112233": {"nom": "Awa", "points": 120},
       "781234567": {"nom": "Moussa", "points": 40},
   }
   ```

   Écrire `peut_utiliser(telephone)` qui **renvoie** `True` si le client a au moins 100 points, `utiliser_points(telephone)` qui retire 100 points et **renvoie** 1000 (la réduction), et `ajouter_points(telephone, montant)` qui ajoute les points gagnés et **renvoie** leur nombre.

4. **Fichier** `superette/rapport.py`

   Importer `date` depuis `datetime`, et `a_commander` avec un import relatif. Définir une liste vide `VENTES`, puis écrire `enregistrer_vente(montant)` qui ajoute le montant à `VENTES`, et `afficher_rapport()` qui affiche la date du jour, le nombre de clients, le chiffre d'affaires, le panier moyen (division entière `//`) et les produits à commander.

5. **Fichier** `main.py`

   Écrire les imports depuis le paquet : le module `stock` entier avec `from superette import stock` ; `ajouter_article`, `total` et `afficher_ticket` depuis `superette.caisse` ; le module `fidelite` avec l'alias `fd` (`from superette import fidelite as fd`) ; `enregistrer_vente` et `afficher_rapport` depuis `superette.rapport`. Puis :

   a) demander `Nouveau client ? (o/n) :` ; tant que la réponse est `o`, servir un client (étapes b à e), puis redemander ;

   b) afficher le stock, créer un panier vide, puis demander produit et quantité jusqu'à ce que le caissier tape `fin`, en appelant `ajouter_article` ;

   c) afficher le ticket et calculer le montant ;

   d) demander le numéro de carte fidélité (Entrée si aucune). Si le numéro est une clé de `fd.CARTES` : si le client a assez de points, lui proposer la réduction et, s'il accepte, la retirer du montant ; puis ajouter ses points et afficher combien il en a gagné ;

   e) afficher le montant à payer et l'enregistrer avec `enregistrer_vente` ;

   f) à la fin de la journée, appeler `afficher_rapport()`.

Exemple d'exécution attendu (ce que tape l'utilisateur est écrit après les deux-points) :

```
=== Supérette Baraka ===
Nouveau client ? (o/n) : o
--- Produits en rayon ---
riz : 4500 FCFA (20 en stock)
(... autres produits ...)
Produit (fin pour terminer) : riz
Quantité : 2
Produit (fin pour terminer) : pates
Quantité : 6
Stock insuffisant : il reste 4 pates
Produit (fin pour terminer) : huile
Quantité : 1
Produit (fin pour terminer) : biscuit
Quantité : 1
Produit inconnu
Produit (fin pour terminer) : fin
------ TICKET ------
2 x riz : 9000 FCFA
1 x huile : 1600 FCFA
Total : 10600 FCFA
Numéro de carte fidélité (Entrée si aucune) : 771112233
Utiliser 100 points pour 1000 FCFA de réduction ? (o/n) : o
Awa gagne 9 points (total : 29)
Montant à payer : 9600 FCFA
Nouveau client ? (o/n) : o
(... stock mis à jour : riz 18, huile 11 ...)
Produit (fin pour terminer) : lait
Quantité : 2
Produit (fin pour terminer) : fin
------ TICKET ------
2 x lait : 5500 FCFA
Total : 5500 FCFA
Numéro de carte fidélité (Entrée si aucune) :
Montant à payer : 5500 FCFA
Nouveau client ? (o/n) : n
===== Rapport du 27/10/2026 =====
Nombre de clients : 2
Chiffre d'affaires : 15100 FCFA
Panier moyen : 7550 FCFA
Produits à commander : ['lait', 'pates']
```